

C# Coding Conventions

Guía de estándares
Smart Cities & Mobility

2025-08-05

1.9



Fecha	Responsable	Cambio	Versión
2016	SIO	Versión inicial	1.0
2020-09-04	JS	Revisión del documento, correcciones, copiado a dev de IoT & Smart Cities	1.1
2022-08-17	JS	Update y revisión previa a compartir con grupo PIS 2022	1.2
2023-07-24	JS	Revisión mitad de año	1.3
2023-08-07	DA	Agrego estándares en secciones de back y front end	1.4
2023-08-08	DA	Agrego estándares en secciones de back y front end	1.5
2023-08-15	JS	Update y revisión previa a compartir con grupo PIS 2023	1.6
2023-08-23	DA	Agrego estándares para retorno de errores	1.7
2025-01-13	JS	Actualización general	1.8
2025-08-05	DA	Se agregó el uso de SnackService con el manejo de errores	1.9

Contenido

Introducción.....	4
Introducción.....	4
Guía.....	4
Guía.....	4
Cadena de caracteres.....	4
Cadena de caracteres.....	4
Enteros.....	4
Enteros.....	4
Booleanos.....	4
Booleanos.....	4
Inicialización de variables.....	4
Inicialización de variables.....	4
Convenciones de nomenclatura de nombres.....	4
Convenciones de nomenclatura de nombres.....	4
Excepciones.....	5
Excepciones.....	5
Registro de mensajes.....	5
Registro de mensajes.....	5
Dispose() & Close().....	5
Dispose() & Close().....	5
Condicionales y LINQ.....	5
Condicionales y LINQ.....	5
Ordenamiento.....	6
Ordenamiento.....	6
Diseño.....	6
Diseño.....	6
Diseño detallado.....	7
Diseño detallado.....	7
Previniendo o encontrando errores.....	7
Previniendo o encontrando errores.....	7
Cultura, idioma, colores.....	7
Cultura, idioma, colores.....	7
Comunicación con Base de Datos.....	7
Comunicación con Base de Datos.....	7

Manejo de Snackbar y errores.....	7
Manejo de Snackbar y errores.....	7
Estándar para Blazor y REST API.....	9
Estándar para Blazor y REST API.....	9
Estándar para Back.....	9
Estándar para Back.....	9
Sugerencias de revisión.....	10
Sugerencias de revisión.....	10
Anexos.....	10
Anexos.....	10
Manejo de Snackbar y errores.....	10
Manejo de Snackbar y errores.....	10

Introducción

Esta guía aplica como criterio general, en cada caso pueden existir excepciones.

Siempre que algo NO se entienda, NO se encuentre en este doc o casos similares, hablarlo y agregarlo.

Guía

Se base en la guía oficial de Microsoft: <https://docs.microsoft.com/en-us/dotnet/csharp/fundamentals/coding-style/identifier-names> y <https://docs.microsoft.com/en-us/dotnet/csharp/programming-guide/inside-a-program/coding-conventions> Se base en la guía oficial de Microsoft: [Identifier-Names](#) y [Coding-Conventions](#)

Cadena de caracteres

- Para definir una cadena de caracteres utilizar el alias string de System.String
- Se utiliza string.Empty para asignar un string vacío
- Se utiliza string.Equals para comparar strings
- Se utiliza string.IsNullOrEmpty para verificar si el string es null o vacío
- Se utiliza StringBuilder para modificar strings que se modifiquen en o(n) o mayor (o desconocido)
- Se utiliza string interpolation para armar string en base a otros componentes

Enteros

- Para definir un entero utilizar el alias int de System.int32
- Para convertir a un entero utilizar int.Parse
- Para valores potencialmente grandes o que puedan crecer se utiliza long

Booleanos

- Para definir un booleano utilizar el alias bool de System.Boolean
- Para convertir a un booleano utilizar bool.Parse

Inicialización de variables

- Iniciar siempre las variables de tipos simples
- Tener en cuenta la característica de nullable de C#

Convenciones de nomenclatura de nombres

- Utilizar la convención UpperCamelCase para describir una clase, un método o propiedad
- Utilizar la convención UpperCamelCase antecedita por 'I' para definir una interfaz

- Utilizar la convención lowerCamelCase para describir una variable local, variable pública o parámetros de entrada de método/función
- Utilizar la convención lowerCamelCase antecedita por ‘_’ para describir una variable privada
- Los nombres de los proyectos, clases, métodos, variables, propiedades, parámetros de entrada de método/función y regiones se escriben en inglés
- Los nombres de los proyectos, clases, métodos, variables y propiedades deben ser explicativos y no referir a tipos de datos
- Los nombres de los proyectos, clases, métodos, variables y propiedades **no** deben utilizar abreviaciones
- Las propiedades de un objeto deben estar escritas como NombreDelObjetoPropiedad
- Usar Delete o Remove según corresponda (delete: de elimina un objeto de forma permanente, remove: se elimina un objeto de un contenedor)
- Usar Create o Add según corresponda (create: se crea un objeto, add: se agrega un objeto a un contenedor)
- Utilizar el nombre Update cuando se editan datos de un objeto

Excepciones

- Utilizar try-catch o try-catch -finally en todo código donde sea necesario
- Los bloques catch deben registrar el error utilizando el LogError adecuado, registrar en la base de datos o visor de eventos
- En el bloque finally se debe agregar lógica para que todo quede consistente en caso de excepción
- Si en un bloque catch se quiere preservar el stack original se utiliza *throw*; (no usar: “*throw exception*”)

Registro de mensajes

- Debe existir un único método para el guardado de mensajes (LogInformation, LogDebug, LogWarning y LogError que re-implementa el de MS)
- Todas las acciones importantes se deben guardar usando el Log anterior o en base de datos o visor de eventos utilizando el método mencionado
- Todos los mensajes deben estar definidos como constantes en una o varias clases (Language, StringResource, etc.)
- Los mensajes deben expresar adecuadamente la acción o el error

Dispose() & Close()

- Se debe invocar estas funciones cuando son ofrecidas (esto puede sustituirse utilizando using, **preferible siempre usar using**)

Condicionales y LINQ

- Si se evalúan condiciones booleanas no especificar “true” o “false”
- No asignar en comparaciones

- No evaluar cada vez en una iteración un método que es fijo
- Al utilizar LINQ usar las expresiones lambda

Ordenamiento

- Dentro de una clase, estructura o interfaz, utilizar el siguiente orden:
 - Constant Fields
 - Fields
 - Constructors
 - Finalizers (Destructors)
 - Delegates
 - Events
 - Enums
 - Interfaces
 - Properties
 - Indexers
 - Methods
 - Structs
 - Classes
- Dentro de los grupos definidos en el punto anterior, utilizar el siguiente orden:
 - public
 - internal
 - protected internal
 - protected
 - private
- Dentro de los grupos definidos en el punto anterior, utilizar el siguiente orden:
 - static
 - non-static
- Y dentro de estos:
 - readonly
 - non-readonly
- Otra cosa que no entre dentro de estos puntos: definir un orden y respetarlo siempre

Diseño

- Al crear un proyecto pensar si tiene sentido que exista el proyecto o alcanza con una clase dentro de otro
- Los proyectos deben tener bajo acoplamiento y alta cohesión (enfocarse en el principio de Single responsibility)
- Los proyectos deberían seguir algún patrón de diseño siempre y cuando ayude a la implementación y entendimiento del proyecto
- Utilizar partial class para subdividir clases que deban ser “grandes” (no debería suceder)

Diseño detallado

- Las clases deben cumplir con el principio de responsabilidad única “Single Responsibility Principle”
- Especificar una sola clase por archivo
- Utilizar ~~#region~~ para separar los grupos definidos en la sección Ordenamiento (no debería ser necesario para dividir “funcionalidades” dentro de una misma clase)
- No debe existir código duplicado, re-utilizar siempre
- Todos los métodos deben estar referenciados
- La cantidad de parámetros en un método debe ser menor o igual a 5 (salvo casos particulares)
- Se debe entender el objetivo de cada método (menos comentarios más entendimiento en el código)
- El método debe tener una única responsabilidad
- La indentación debe ser la adecuada
- Utilizar siempre {} para todo (especial énfasis en bloques if con una sola línea)

Previendo o encontrando errores

- No se deben visualizar errores ni warnings en la ventana Output

Cultura, idioma, colores

- La implementación debe ser independiente de la cultura (CultureInfo)
- Debe utilizarse LanguageResource para todos los textos que se muestren en pantalla
- La paleta de colores debe estar definida en un único lugar

Comunicación con Base de Datos

- Los nombres de los parámetros que se pasan a un Store Procedure o que se usan en las queries parametrizadas se definen como una propiedad NombreDelObjetoPropiedad

Manejo de Snackbar y errores

Se debe de implementar la siguiente class (el código se encuentra en la sección de anexos):

```

99+ references | Darwin Araujo Moraes, 102 days ago | 1 author, 1 change
public class SnackBarService
{
    private ISnackBar _snackbar;
    0 references | 0 changes | 0 authors, 0 changes
    public SnackBarService(ISnackBar snackbar)
    {
        _snackbar = snackbar;
    }

    65 references | Darwin Araujo Moraes, 102 days ago | 1 author, 1 change
    public async Task<SnackBar?> Add(HttpResponseMessage response)
    {
        string? resCode = await response.Content.ReadAsStringAsync();
        string? error = null;
        error = Language.ResourceManager.GetString(resCode);
        if (string.IsNullOrEmpty(error))
        {
            error = Language.ExceptionMessage;
        }
        return _snackbar.Add(error, Severity.Error);
    }

    #region MudBlazor base SnackBar methods

    // TOCHECK: No se implemento porque no se esta usando actualmente, a futuro si se precisa se podria implementar
    //public event Action OnSnackbarsUpdated;

    0 references | 0 changes | 0 authors, 0 changes
    public IEnumerable<SnackBar> ShownSnackbars => _snackbar.ShownSnackbars;

    0 references | 0 changes | 0 authors, 0 changes
    public SnackBarConfiguration Configuration => _snackbar.Configuration;

    99+ references | Darwin Araujo Moraes, 102 days ago | 1 author, 1 change
    public SnackBar? Add(string message, Severity severity = Severity.Normal, Action<SnackBarOptions>? configure = null, string key = "")
        => _snackbar.Add(message, severity, configure, key);

    0 references | Darwin Araujo Moraes, 102 days ago | 1 author, 1 change
    public SnackBar? Add(RenderFragment message, Severity severity = Severity.Normal, Action<SnackBarOptions>? configure = null, string key = "")
        => _snackbar.Add(message, severity, configure, key);

    0 references | 0 changes | 0 authors, 0 changes
    public void Clear() => _snackbar.Clear();

    0 references | 0 changes | 0 authors, 0 changes
    public void Dispose() => _snackbar.Dispose();

    0 references | 0 changes | 0 authors, 0 changes
    public void Remove(SnackBar snackbar) => _snackbar.Remove(snackbar);

    0 references | 0 changes | 0 authors, 0 changes
    public void RemoveByKey(string key) => _snackbar.RemoveByKey(key);
    #endregion
}

```

Se debe de registrar la class en Program.cs del cliente:

```

// SnackBarService personalizado
builder.Services.AddScoped<SnackBarService>();

```

Los errores del servidor se deben de enviar de la siguiente forma, en donde se debe de mandar el nombre del string resource que se debe utilizar para mostrar el mensaje:

```

return BadRequest(nameof(Language.InvalidData));

```

Para mostrar el error en el cliente primero se debe de hacer inject de SnackBarService:

```

@Inject SnackBarService SnackBar

```

Para mostrar mensajes de error se debe pasar la respuesta generada por el servidor:

```

await SnackBar.Add(response);

```

Para mostrar mensajes generales se debe pasar el texto y la severidad:

```

SnackBar.Add(Language.ExceptionMessage, Severity.Error);

```


Estándar para Blazor y REST API

Nombre	Descripción	Ejemplos
GET	No puede cambiar el estado en back, solo para obtener datos	
POST	Crear nuevos recursos, existen excepciones a discutir	
PUT	Actualizar recursos existentes	
DELETE	Eliminar recursos	
Formato de llamado	Se debe de seguir el siguiente formato para realizar llamados: api/metodo/ID	PUT: url/api/roles/4 [role] Actualiza el rol de id 4 con los datos del role
Etiquetas sin contenido	Si una etiqueta no tiene contenido no se debe utilizar etiqueta de cierra	Bien: <Etiqueta>Contenido</Etiqueta> Mal: <Etiqueta> </Etiqueta>
Ubicación Injects	Los Inject se deben de realizar en el archivo .razor.cs, no en el .razor	
Sin usings inesesarios	Los usings que no son referenciados se deben de borrar	
Orden de los @	Se debe respetar el siguiente orden en los .razor @page => @attribute => @using => @inject	
Orden alfabético en Injects	Los injects deben de estar ordenados en orden alfabético acendente	
Orden parámetros al usar componente	Si el componente define un parámetro T o context, que estos siempre sean asignados primero en el componente	<ComponenteN T="TType" Context="context" ... />

Estándar para Back

Nombre	Descripción	Ejemplos
Instancia de listas	Cuando la instanciación es directa (no es abstracta o interface) usamos directo new()	Bien: List<string> strings = new(); Mal: List<string> strings = new List<string>();
Etiqueta Route de controllers	La etiqute de be utilizar "[Controller]" no el nombre del controlador	Bien: [Route("api/[Controller]")] Mal: [Route("api/users")]
return Error 500	Usar 500InternalServerError para excepción no controlada	
return	Usar BadRequest(<número negativo>)	

BadRquest	para errores en las entradas u otros errores, los números negativos se utilizarán en caso de que se desee distinguir errores	
return NotFound	NotFound(<número negativo>) para cuando no existe un elemento en la DB, los números negativos se utilizarán en caso de que se desee distinguir errores	
return Conflict	Conflict(número negativo) se usara cuando una la acción no mantiene la consistencia de los datos, los números negativos se utilizarán en caso de que se desee distinguir errores	borrar algo con relaciones, asignar algo a algo q ya esta tiene algo asignado etc

Sugerencias de revisión

Nombre	Descripción	Ejemplos
Revisar permisos en .razor	Cada .razor se debe de verificar que tenga el permiso correspondiente	
Variables booleanas de estados sin is	Las variables booleanas que representen un estado en el sistema no deben de comenzar con is	Bien: bool showPassword Mal: isShowPassword
Parse para convertir strings	Para convertir strings a otros datos utilizar Parse, se puede usar TryParse para verificar si es posible (retorna bool)	Int.Parse(str)

Anexos

Manejo de Snackbar y errores

```
public class SnackService
{
    private ISnackbar _snackbar;
    public SnackService(ISnackbar snackbar)
    {
```

```

        _snackbar = snackbar;
    }

    public async Task<Snackbar?> Add(HttpResponseMessage response)
    {
        string? resCode = await response.Content.ReadAsStringAsync();
        string? error = null;
        error = Language.ResourceManager.GetString(resCode);
        if (string.IsNullOrEmpty(error))
        {
            error = Language.ExceptionMessage;
        }
        return _snackbar.Add(error, Severity.Error);
    }

    #region MudBlazor base Snackbar methods

    public IEnumerable<Snackbar> ShownSnackbars => _snackbar.ShownSnackbars;

    public SnackbarConfiguration Configuration => _snackbar.Configuration;

    public Snackbar? Add(string message, Severity severity = Severity.Normal,
        Action<SnackbarOptions>? configure = null, string key = "")
        => _snackbar.Add(message, severity, configure, key);

    public Snackbar? Add(RenderFragment message, Severity severity = Severity.Normal,
        Action<SnackbarOptions>? configure = null, string key = "")
        => _snackbar.Add(message, severity, configure, key);

    public void Clear() => _snackbar.Clear();

    public void Dispose() => _snackbar.Dispose();

    public void Remove(Snackbar snackbar) => _snackbar.Remove(snackbar);

    public void RemoveByKey(string key) => _snackbar.RemoveByKey(key);
    #endregion
}

```